

DOCKER: WHY, HOW, AND WHEN

Jarret Fossen

TWO PROBLEMS WITH APPLICATION ENVIRONMENTS

1. Environment inconsistency:

- “It works on my machine.”
- Different runtimes, libraries, or settings can make the same code work on one machine and fail on another.

2. Dependency conflicts:

- Applications sharing an environment may need incompatible versions of the same software.
- Updating a dependency for one application can break another.

Both problems stem from applications relying on environments that vary across machines or are shared with other applications

How Docker Helps

Consistent environments across machines

- Your laptop, a teammate's laptop, and the server run the same image with the same application dependencies.

Isolated dependencies on the same machine

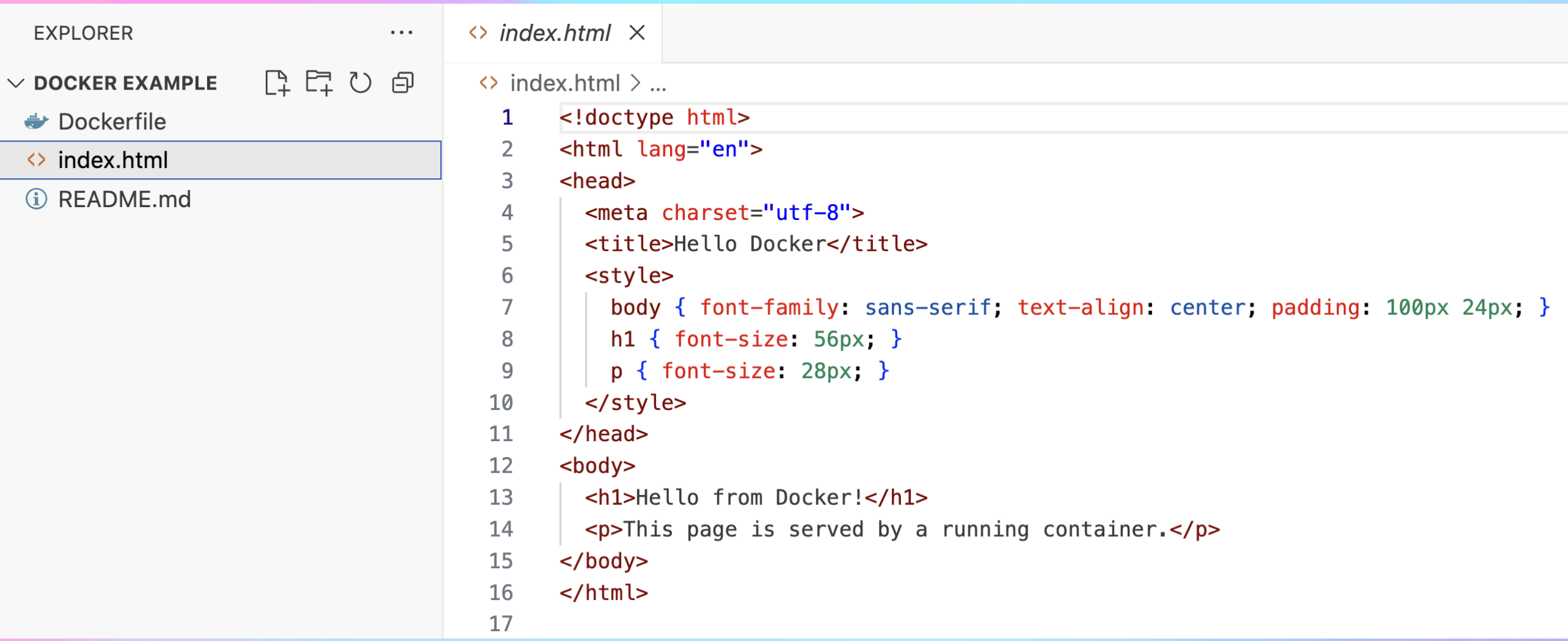
- Service A uses Python 3.10 while Service B uses Python 3.12, each in its own container.

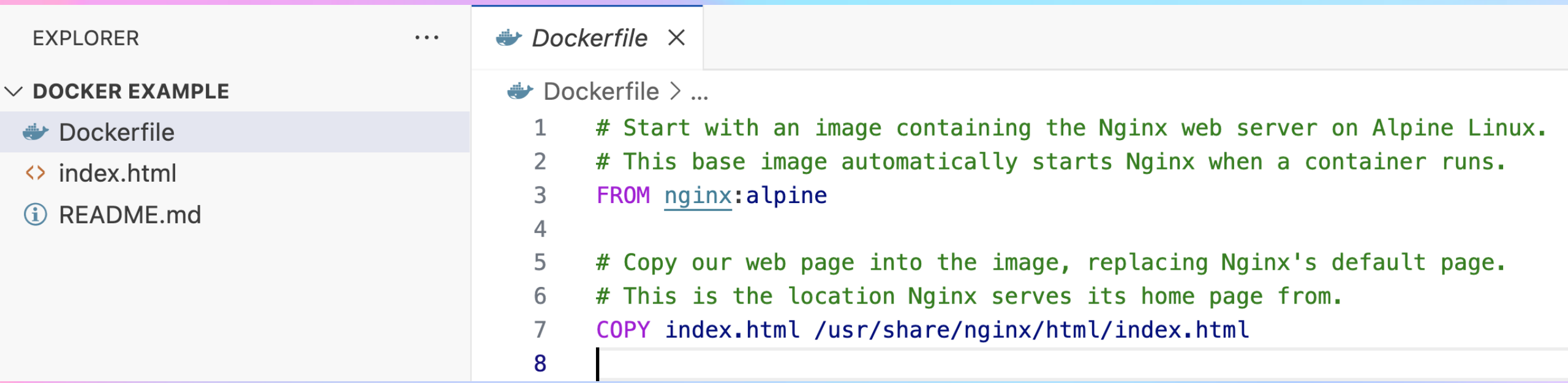
THREE FUNDAMENTAL CONCEPTS

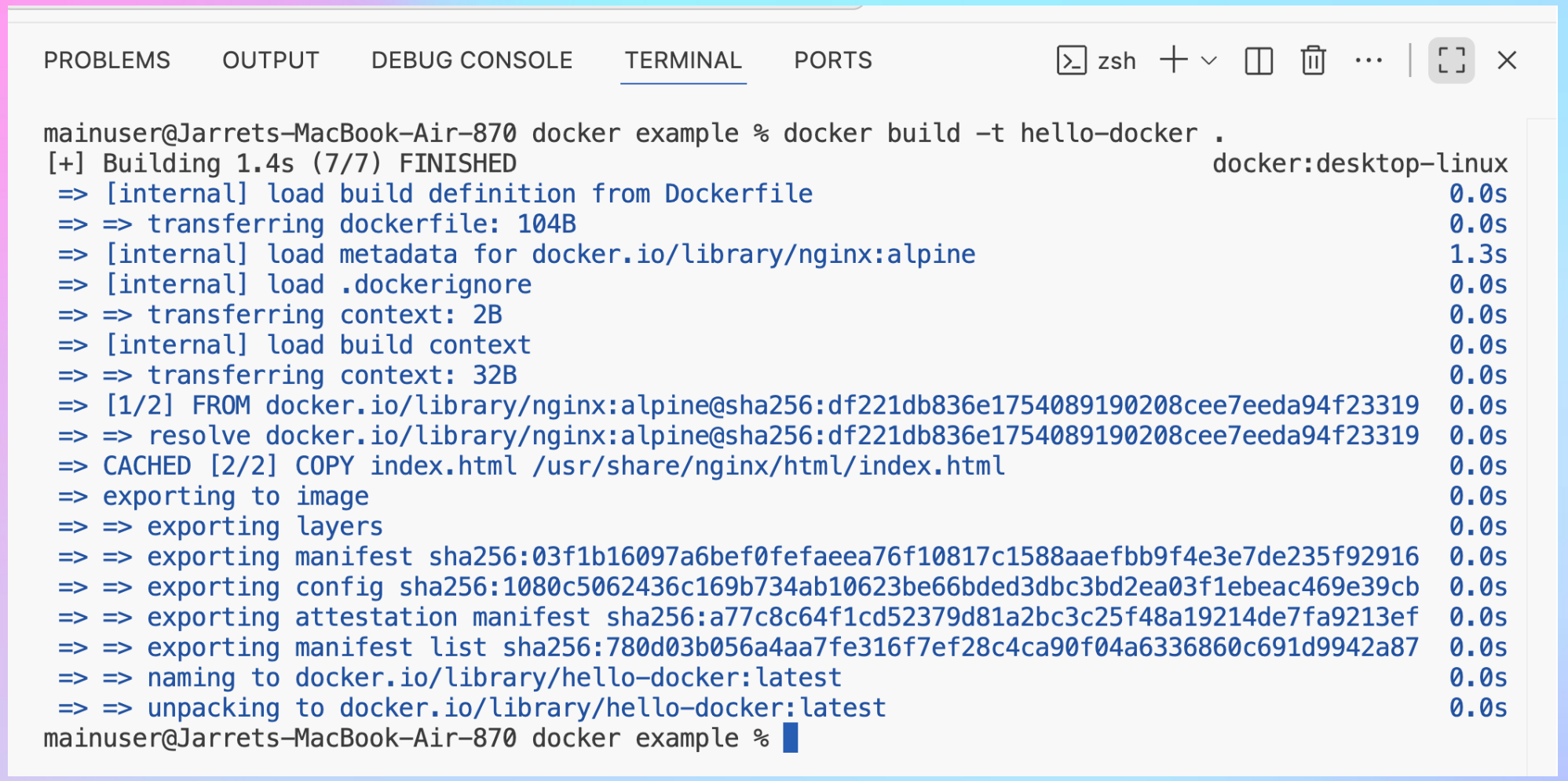
Dockerfile: A text file containing instructions for building a Docker image.

Image: A template containing an application, its runtime (e.g., a Python interpreter), and dependencies (e.g., libraries), used to create containers.

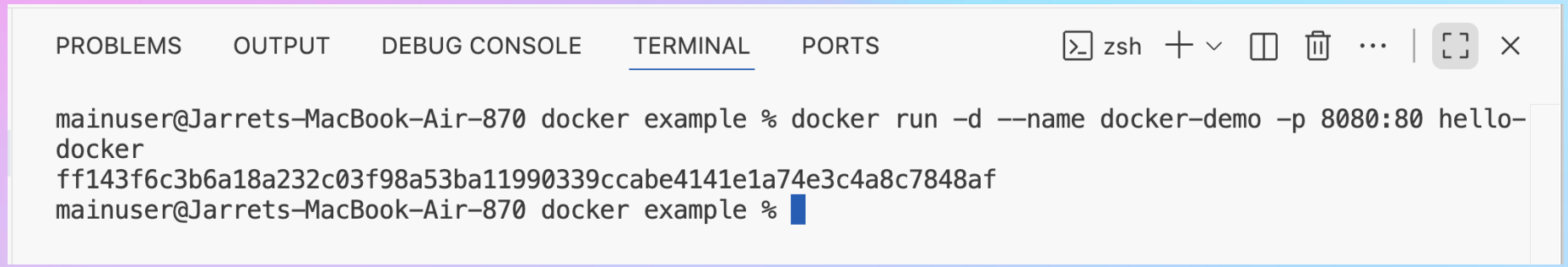
Container: An isolated environment created from an image, in which the application runs.


EXPLORER
DOCKER EXAMPLE
Dockerfile
index.html
README.md
index.html
index.html > ...
1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8">
5 <title>Hello Docker</title>
6 <style>
7 body { font-family: sans-serif; text-align: center; padding: 100px 24px; }
8 h1 { font-size: 56px; }
9 p { font-size: 28px; }
10 </style>
11 </head>
12 <body>
13 <h1>Hello from Docker!</h1>
14 This page is served by a running container.
15 </body>
16 </html>
17


EXPLORER
DOCKER EXAMPLE
Dockerfile
index.html
README.md
Dockerfile
Dockerfile > ...
1 # Start with an image containing the Nginx web server on Alpine Linux.
2 # This base image automatically starts Nginx when a container runs.
3 FROM nginx:alpine
4
5 # Copy our web page into the image, replacing Nginx's default page.
6 # This is the location Nginx serves its home page from.
7 COPY index.html /usr/share/nginx/html/index.html
8



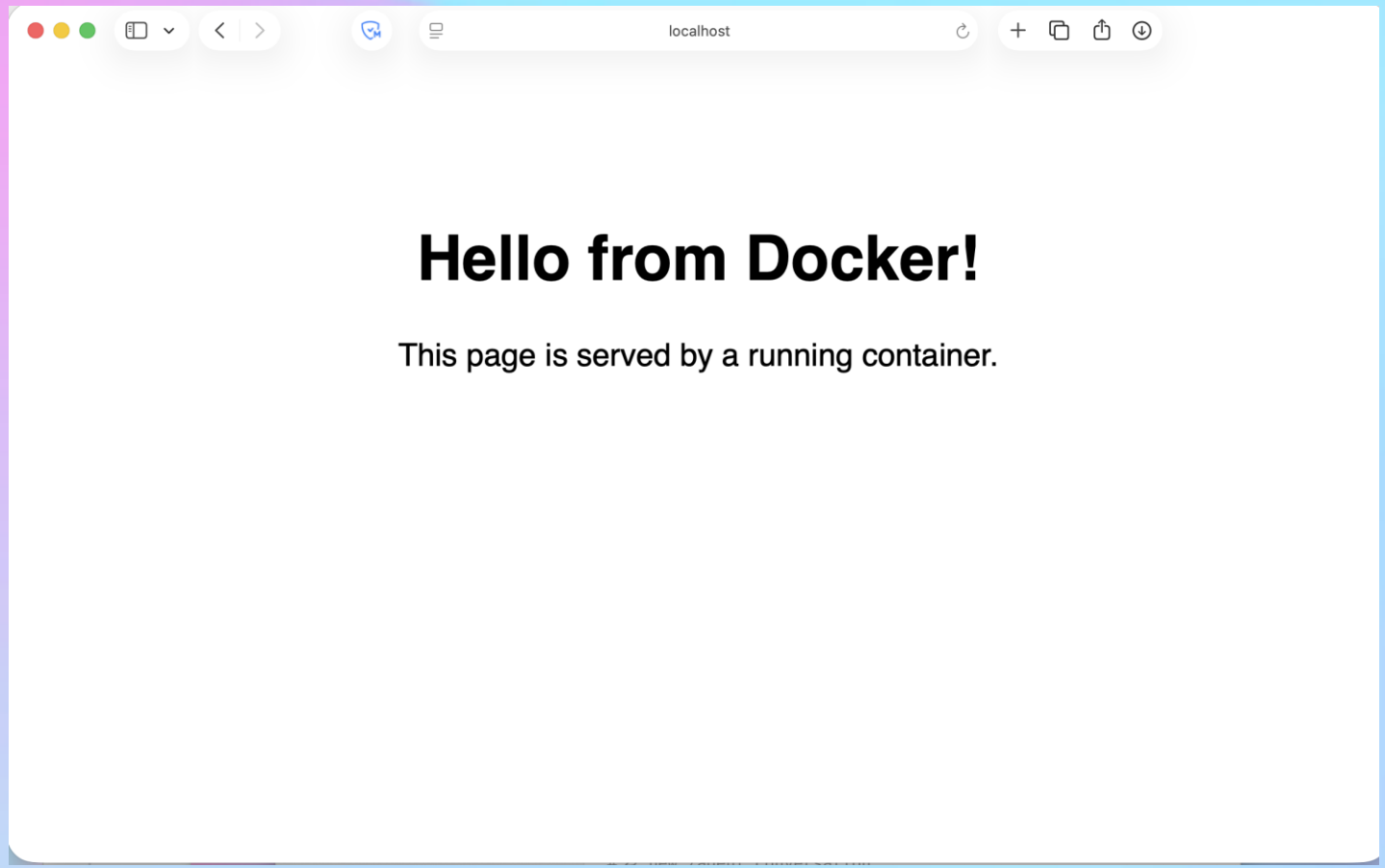
Docker follows instructions from Dockerfile to build an image called hello-docker.
This image contains the web server and our page.



Now we create a container from that image. It runs in the background, and we connect port 8080 on my machine to port 80 inside the container.

We give the container the name "hello-docker". And we run it in the background with -d

Success!



When I visit localhost on port 8080, Nginx running inside the container serves our page. I didn’t need to install Nginx directly on my Mac.

CONTAINERS VS. VIRTUAL MACHINES

Containers:

- OS-level virtualization: Each container has an isolated environment while sharing the underlying kernel.
- Can have different tools, libraries, and Linux distribution files.
- Generally start faster and use less memory than VMs.

VMs:

- Hardware virtualization: Each VM runs its own guest OS, including its own kernel
- Generally require more memory and startup time because each runs a full guest OS.

UNDER THE HOOD

Docker builds on Linux kernel features:

- Namespaces isolate what processes can see, such as other processes, networks, and filesystem mounts.
- Control groups (cgroups) track resource usage and enforce configured CPU and memory limits.

Where do Linux containers run?:

- **On Linux:** Docker Engine can run containers directly, sharing the host's Linux kernel.
- **On macOS and Windows:** Docker Desktop runs Linux containers in a Linux VM. The containers share that VM's kernel.

PRACTICAL CONSIDERATIONS

THE END!

REFERENCES

Docker, Inc. “What is a container?” Docker Docs.
https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

Docker, Inc. “What is Docker?” Docker Docs.
https://docs.docker.com/get-started/docker-overview/

Docker, Inc. “Docker Desktop license agreement.” Docker Docs.
https://docs.docker.com/subscription-billing/desktop-license/